TRAMS: Training-free Memory Selection for Long-range Language Modeling

Haofei Yu[♡*], **Cunxiang Wang**[♣†], **Yue Zhang**[♣], **Wei Bi**[♢‡]
[♡]Language Technologies Institute, Carnegie Mellon University, USA
[♣]School of Engineering, Westlake University, China [♢] Tencent AI Lab, China
haofeiy@cs.cmu.edu, {wangcunxiang, zhangyue}@westlake.edu.cn,
victoriabi@tencent.com

arXiv:2310.15494v3 [cs.CL] 20 Dec 2023

Abstract

The Transformer architecture is crucial for numerous AI models, but it still faces challenges in long-range language modeling. Though several specific transformer architectures have been designed to tackle issues of long-range dependencies, existing methods like Transformer-XL are plagued by a high percentage of ineffective memories. In this study, we present a plug-and-play strategy, known as **TRA**ining-free **M**emory **S**election (TRAMS), that selects tokens participating in attention calculation based on one simple metric. This strategy allows us to keep tokens that are likely to have a high attention score with the current queries and ignore the other ones. We have tested our approach on the word-level benchmark (WikiText-103) and the character-level benchmark (enwik8), and the results indicate an improvement without having additional training or adding additional parameters.

1 Introduction

Transformer-based models (Kenton and Toutanova, 2019; Liu et al., 2019; Raffel et al., 2020; Lan et al., 2019; Brown et al., 2020) have achieved remarkable performance over the past few years. The key component of these model architectures is the attention mechanism (Vaswani et al., 2017). However, the original attention design struggles to efficiently handle long sequences, which becomes particularly problematic in scenarios such as document-level translation (Werlen et al., 2018; Kim et al., 2019) and large-scale text generation (Zhou et al., 2023), as its time and space computation costs increase quadratically with the sequence length (Tay et al., 2022). The primary factor for this elevated computational complexity can be traced back to the multiplication between queries and keys used in the attention module. In general, the time complexity for calculation is $\mathcal{O}(N^2d)$ if a transformer model with d dimensions is set up with an input consisting of N tokens.

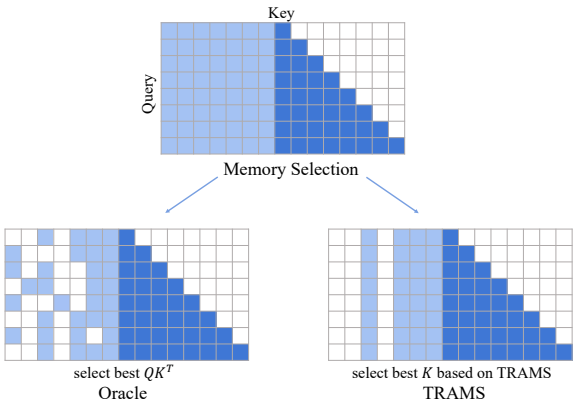

Figure 1: Two memory selection methods: For oracle, it selects memories with the highest attention scores after computing $QK^\top$. For TRAMS, it selects important key/value pairs that are independent of queries based on our self-defined metric before computing $QK^\top$.

To tackle this computation bottleneck, numerous efforts have been made. The first line of work is to find a new efficient expression to compute the attention score. Despite the advancements made, these methods often compromise performance, thus paving the way for alternative solutions. Efficient architectures that provide an approximate expression of attention have been explored widely (Wang et al., 2020; Peng et al., 2022b,a; Choromanski et al., 2021; Zheng et al., 2022b,a). The second line of work is to keep the calculation expression the same and use an external structure like hash function (Kitaev et al., 2019; Daras et al., 2020), clustering (Roy et al., 2021; Vyas et al., 2020) and memory selector (Pietruszka et al., 2022; Dai et al., 2019; Bertsch et al., 2023; Sukhbaatar et al., 2021, 2019; Child et al., 2019) to find the suitable subset of queries and keys in the long sequence for attention calculation.

[*]Work done during internship at Tencent AI Lab.
[†]Co-first Author.
[‡]The correponding author.

Our work falls into the second category, in which we propose a training-free memory selection mechanism to select suitable tokens for attention computation. Specifically, we focus on pushing Transformer-XL (Dai et al., 2019) architecture to a better position by selecting higher-quality tokens inside its memory. Based on our initial investigation, we construct a memory subset by selecting 50% of the memories with the largest attention values and maintaining the same performance. It indicates that **a large portion of information in memory is not fully utilized**. This motivates us to explore better methods to optimize memory usage.

Illustrated in Figure 1, we propose a **TRA**ining-free **M**emory **S**election method (`TRAMS`) that can be directly plugged into memory-based long-range language models and reduces the time complexity of computing attention matrix. Through experiments on two language modeling benchmark datasets, namely word-level `WikiText-103` (Merity et al., 2016) and character-level `enwik8` (Mahoney, 2011), we achieve an improvement in the model's performance, as demonstrated by a 0.19 perplexity (ppl) drop in `WikiText-103` and a 0.017 reduction in bits-per-character (bpc) in `enwik8`.

To our knowledge, we are the first to design a training-free memory selection method based on Transformer-XL architecture.[1]

2 Method

2.1 Problem Definition

We use $\boldsymbol{h} \in \mathbb{R}^{N\times d}$ to represent the input hidden states for the attention module, $\boldsymbol{o} \in \mathbb{R}^{N\times d}$ to represent the output hidden states for the attention module, $\boldsymbol{m} \in \mathbb{R}^{M\times d}$ to represent the memory hidden states used in the attention calculation. We use W_Q, W_K, W_V to represent the trainable projection matrix in the attention module. We define d for the dimension of the model, M for the memory size, and N for the input size. The attention calculation process can be formally written as $\boldsymbol{o} = \text{Attn}(\boldsymbol{h}, \boldsymbol{m})$.

With the above annotations, the problem of memory selection can be defined as choosing a subset of hidden states memory $\tilde{\boldsymbol{m}}$ from the memory $\boldsymbol{m}$ that brings the minimum difference to the transformer layer output but with a smaller memory size.

$$\tilde{\boldsymbol{m}}^* = \arg\min_{\tilde{\boldsymbol{m}}\subset\boldsymbol{m}} \|\text{Attn}(\boldsymbol{h}, \tilde{\boldsymbol{m}}) - \text{Attn}(\boldsymbol{h}, \boldsymbol{m})\| \quad (1)$$

[1]Source code for this paper is available at https://github.com/lwaekfjlk/TRAMS.

2.2 Attention Reformulation

Standard Attention In a memory-augmented language model, the standard attention mechanism (Vaswani et al., 2017) between input hidden states and memory hidden states can be written as:

$$\text{Attn}(\boldsymbol{h}, \boldsymbol{m}) = \text{softmax}(\frac{QK^\top}{\sqrt{d}})V \quad (2)$$

where $Q = \boldsymbol{h}W_Q$ is the product of target token hidden states $\boldsymbol{h}$ and query projection matrix W_Q; $K = \boldsymbol{m}W_K$ is the product of memory token hidden states $\boldsymbol{m}$ and key projection matrix W_K; $V = \boldsymbol{m}W_V$ is also the product of memory token hidden states $\boldsymbol{m}$ and value projection matrix W_V.

Unlimiformer Attention Different from the well-known attention score calculation, Unlimiformer (Bertsch et al., 2023) proposed a rewritten way to compute the dot-product part of cross-attention in the encoder-decoder architecture:

$$\begin{aligned} QK^\top &= (\boldsymbol{h}_d W_Q)(\boldsymbol{h}_e W_K)^\top \\ &= (\boldsymbol{h}_d W_Q W_K^\top)\boldsymbol{h}_e^\top \end{aligned} \quad (3)$$

where $\boldsymbol{h}_e$ is the encoder hidden state and $\boldsymbol{h}_d$ is the decoder hidden state. It allows Unlimiformer to avoid indexing the keys for each head and layer separately and avoid storing values in a separate index from the keys during kNN-based searching and retrieval stage, making it more efficient.

`TRAMS` Attention Even though we have no need to store or index any key or value for our method, Unlimiformer attention motivates us to transfer more useful information to keys by reformulating attention and allows us to do more effective memory selection solely based on reformulated keys. We can compute this attention formula in a different order but maintain the same result:

$$\begin{aligned} QK^\top &= (\boldsymbol{h}W_Q)(\boldsymbol{m}W_K)^\top \\ &= (\boldsymbol{h})(\boldsymbol{m}W_K W_Q^\top)^\top \end{aligned} \quad (4)$$

Thus, we define $Q' = \boldsymbol{h}$ as the reformulated query for this attention expression and $K' = \boldsymbol{m}W_K W_Q^\top$ as the reformulated keys for attention. With this reformulation, we transfer all attention-related parametric information onto reformulated key vectors.

2.3 Transformer Hidden Space

Since $\boldsymbol{h}$ is the input of the current transformer layer and also the output of the previous transformer layer, it is the result of the last layer's `Layernorm`

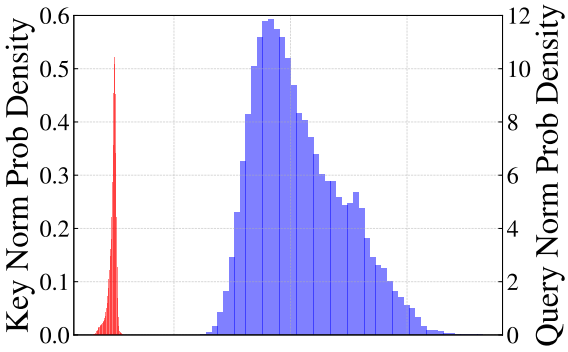

Figure 2: Norm distribution of reformulated Q' and K'. The red distribution represents the query norm. The blue distribution represents the key norm.

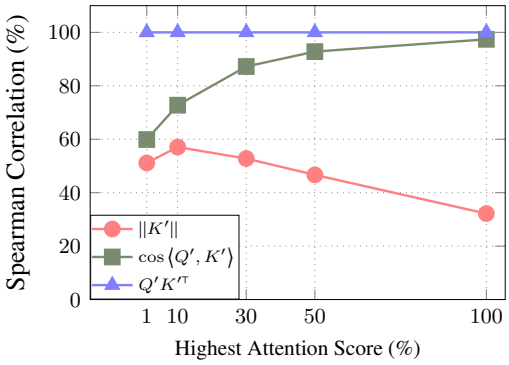

Figure 3: Spearman Correlation Score on different ranking metrics with the groundtruth one.

operation. We can define the coordinate-wise average of $\boldsymbol{h}$ as μ and the coordinate-wise standard deviation of $\boldsymbol{h}$ as σ. Expressions can be written as:

$$\mu = \frac{1}{d}\sum_{i=1}^{d} h_i \approx 0, \quad \sigma = \sqrt{\frac{1}{d}\sum_{i=1}^{d}(h_i - \mu)^2} \approx 1 \quad (5)$$

Since the mean value for the hidden states $\boldsymbol{h}$ is around zero, we can confirm the hidden states vectors are approximately orthogonal to the $\vec{\mathbb{1}}$ vector and the L2 norm of hidden states is around $\sqrt{d}$.

With this approximation, we can expand our reformulated attention score as:

$$\begin{aligned} Q'K'^{\top} &= (\boldsymbol{h})(\boldsymbol{m}W_K W_Q^{\top})^{\top} \\ &= \|Q'\| \cdot \|K'\| \cdot \cos\langle Q', K'\rangle \\ &\approx \sqrt{d} \cdot \|K'\| \cdot \cos\langle Q', K'\rangle \end{aligned} \quad (6)$$

where $\|Q'\|$ stands the L2 norm for Q' and $\|K'\|$ stands for the L2 norm for K'. Based on Fig 2, we see that reformulated query norm $\|Q'\|$ has a much sharper distribution compared with key norm $\|K'\|$, indicating reformulated query norm can be approximated by a constant factor.

2.4 Training-free Memory Selection (TRAMS)

Our target for memory selection is to recover the complete attention score with as few memory tokens as possible. This problem is equivalent to finding the subset of memory tokens that have the highest attention scores with queries. We propose a heuristic method to perform token-level selection for each layer and each head based on a memory-independent metric in this section.

There are two crucial components for calculating the attention score after approximating $\|Q'\|$ with a constant factor: the norm of the reformulated keys $\|K'\|$ and the angles between the reformulated keys and queries $\arccos\langle Q', K'\rangle$, which is proved in Khandelwal et al. (2019). Commonly, we believe that $\arccos\langle Q', K'\rangle$ is the more important factor in general. Yet, if we use the ranking of attention score value for all query and key pairs as ground-truth ranking, based on Fig 3, we empirically discovered that rankings based on key norms and rankings based on angles produce close Spearman correlation scores when only taking the highest 1% attention scores into account. Therefore, it indicates that we can rank our memory tokens based on $\|K'\|$ solely to gain a relatively good performance when we desire top 1% attention scores with queries in our memories instead of all.

Additionally, we discovered that relying solely on a large norm isn't sufficient as a constraint. Specifically, keys that are nearer to $\vec{\mathbb{1}}$ tend to yield a higher attention score. To address this, we introduce a combined metric: $s = \cos\left\langle K', \vec{\mathbb{1}}\right\rangle \|K'\|$. This metric allows us to identify tokens that can produce *high* attention scores when paired with the appropriate query (owing to a high value of $\|K'\|$) and *low* scores when paired with an unsuitable query (owing to the high level of orthogonality with the query space based on $\cos\left\langle K', \vec{\mathbb{1}}\right\rangle$). This is due to the near orthogonality to the query space, as indicated by a small angle with $\vec{\mathbb{1}}$, which is orthogonal to the query space.

3 Experiments

We introduce the compared methods and report the main results and analysis on different attention variants for inference in this section. Datasets details for `WikiText-103` and `enwik8` benchmarks and their evaluation metric details are included in Appendix A. The details of the model that we built memory selection on can be seen in Appendix B.

WikiText-103				
Model	M	m	n	PPL (↓)
Transformer+RPE	-	-	-	29.14
Transformer-XL	-	200	64	24.17
TRAMS	400	200	64	**23.98**
enwik8				
Model	M	m	n	bpc (↓)
Transformer+RPE	-	-	-	1.240
Transformer-XL	-	200	64	1.215
TRAMS	400	200	64	**1.198**

Table 1: Model performance on the word-level WikiText-103 and the character-level enwik8 datasets.

3.1 Compared Methods

Transformer+RPE (Vaswani et al., 2017): the vanilla transformer baseline with relative position embedding that is the same as Transformer-XL. Therefore, the only difference between this model and Transformer-XL is the additional memories. More information related to relative position embedding can be seen in Appendix C.

Transformer-XL (Dai et al., 2019): a specific-designed architecture for long-range language modeling. It includes relative position embedding and recurrent memories per layer. Memory slots are filled with hidden states from previous time steps.

3.2 Experimental Settings

We compare our methods with the Transformer-XL (Dai et al., 2019) under the same size of memory (m = 200) for attention calculation. For the input token length n for both models, we keep the same as in (Dai et al., 2019) (n = 64). Additionally, the memory selection process is performed on a memory pool with the size of M. Our model and the Transformer-XL share the model parameters but have different inference strategies.

3.3 Main Results

The main results of WikiText-103 and enwik8 datasets are shown in Table 1. Without additional training or additional parameters, we gain 0.19 improvement in perplexity and 0.017 improvement for bit-per-character with our TRAMS mechanism. We implement p-test by inferencing on multiple model checkpoints and prove that our results are significant ($p < 0.05$).

4 Discussions

Is TRAMS vulnerable to the selection of hyperparameters? There are three hyper-parameters in TRAMS: the memory pool size M that TRAMS is able to select from; the selected memory size m that is used in the forward process; and the input token size n that is involved in both backward and forward process.

From the ablation study on M, Figure 4 suggests an optimal range between 300 to 400 for the memory pool size. Beyond this range, enlarging the memory pool often leads to the selection of irrelevant tokens, deteriorating our performance. Regarding m, Figure 5 indicates that TRAMS witnesses a substantial drop in perplexity when the memory size selected is about 25%. Selecting a larger portion does not yield further improvement. This is consistent with Figure 3, where TRAMS excels by concentrating on the top 10% of results. Lastly, in the study on n, Figure 6 shows that as the target token length decreases, the efficacy of memory selection improves.

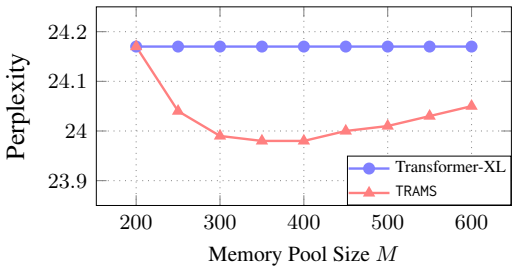

Figure 4: Ablation study on memory pool size M when we fix m=200 and n=64.

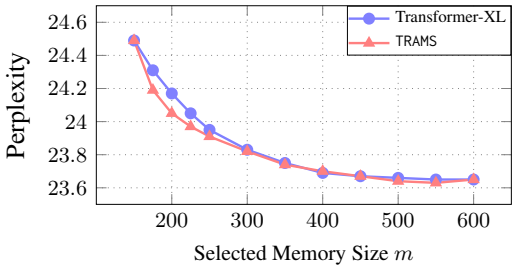

Figure 5: Ablation study on selected memory size m when we fix M=600 and n=64.

What is the inference cost compared to Transformer-XL? Since there is no training part in our model, we focus on discussing the inference cost. Compared with Transformer-XL, our model requires storing a larger memory pool to do memory selection. Therefore, the memory cost of our method would be larger. When it comes to timing cost, our model has an additional memory token

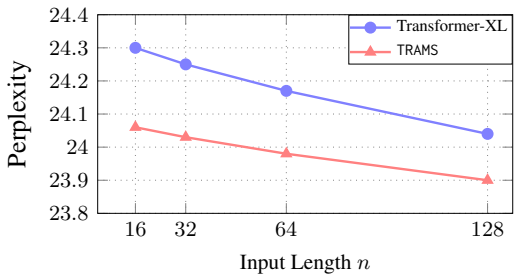

Figure 6: Ablation study on target length n when we fix M=400 and m=200.

Model	Peak GPU Mem (MB)	Wall-clock Time (s)
Transformer-XL	3529	33.27
TRAMS	3719	49.55

Table 2: Results on GPU peak memory usage and wall-clock inference time on WikiText-103.

norm computation memory sorting operations, and memory selection operations for each layer. These extra operations require extra inference time. Table 2 shows the GPU memory cost and wall-clock time for the Transformer-XL baseline and our model. Our model requires slightly more GPU memory usage and around 50% additional inference time for memory selection.

How does TRAMS benefit from memory selection? Memory selection helps the model pick tokens with higher attention scores with the queries, thus increasing the average memory utilization. Quantitatively, our method improves the average attention probability by **24.25**% for the same size of memory compared with Transformer-XL.

Does each layer hold the same importance? Based on Figure 7, we show the ablation study when applying memory selection on each layer while remaining other layers the same. There is an observable drop when we apply the memory selection on the deeper layers starting from Layer 13 while we do not observe a clear influence when applying memory selection on shallow layers.

5 Case Study

To have an understanding of what kind of context should be selected, we provide one example case to understand specifically what kind of tokens in the memory would be selected. Based on Table 3, we can see that most of the selected memory tokens are low-frequency words. Those low-frequency words like "John" in the memory would be beneficial for the prediction of "John" in the target sequence.

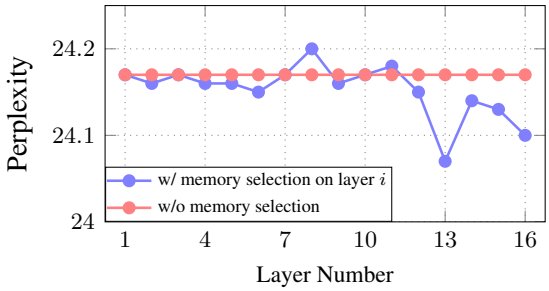

Figure 7: Ablation Study on Layer-wise Importance on WikiText-103.

Memory Sequence Segment
...Simon Stephens, which was performed in 2001 at the Royal Court Theatre. He had a guest role in the television series Judge John Deed in 2002. In 2004 Boulter landed a role as "Craig" in the episode "Teddy's Story" of the television series The Long Firm; he starred alongside actors Mark Strong and Derek Jacobi. He was cast in the 2005 theatre productions of the Philip Ridley play Mercury Fur, which was performed at the Drum Theatre in Plymouth and the <unk> Chocolate Factory in London. He was directed by John Tiffany and starred alongside Ben Whishaw, Shane Zaza, Harry Kent, Fraser Ayres, Sophie Stanton, and Dominic Hall. <eos> In 2006, Boulter starred alongside Whishaw in the play Citizenship written by Mark Ravenhill...
Target Sequence Segment
He appeared in the television series Judge John Deed in 2002 ...

Table 3: Case Study for memory selection from WikiText-103. text indicates that this word in memory sequence is selected and used in the forward pass. text indicates that this word in the target sequence benefits from the memory.

6 Conclusion

In this work, we formulate the problem of memory selection in transformer architecture and reformulate the attention calculation process to obtain our self-defined queries and keys. After that, we propose a query-independent metric that utilizes memory hidden states to implement a training-free memory selector. Our experiments indicate that this method offers a simple yet effective means of identifying valuable memory tokens. Exploring optimal memory selection strategies for large language models is a promising avenue for future research. Additionally, integrating trainable parameters into these models as memory selectors presents another exciting direction for future work.

Limitations

Our study has a couple of main limitations. First, we are currently focusing on the Transformer-XL architecture, but there are many other models with different sizes we haven't tried. It indicates that our findings could be limited to typical transformer architecture. Second, our method has many hyperparameters including M, m, and n. Adjusting them can greatly change how well our model works. A careful calibration is thus necessary, and one must tread cautiously to strike a balance and achieve the desired performance, which could be time-consuming and computationally expensive.

Ethics Statement

There are no recognized potential risks.

References

Amanda Bertsch, Uri Alon, Graham Neubig, and Matthew R Gormley. 2023. Unlimiformer: Long-range transformers with unlimited length input. *arXiv preprint arXiv:2305.01625*.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901.

Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. 2019. Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509*.

Krzysztof Choromanski, Haoxian Chen, Han Lin, Yuanzhe Ma, Arijit Sehanobish, Deepali Jain, Michael S Ryoo, Jake Varley, Andy Zeng, Valerii Likhosherstov, et al. 2021. Hybrid random features. *arXiv preprint arXiv:2110.04367*.

Zihang Dai, Zhilin Yang, Yiming Yang, Jaime G Carbonell, Quoc Le, and Ruslan Salakhutdinov. 2019. Transformer-xl: Attentive language models beyond a fixed-length context. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 2978–2988.

Giannis Daras, Nikita Kitaev, Augustus Odena, and Alexandros G Dimakis. 2020. Smyrf-efficient attention using asymmetric clustering. *Advances in Neural Information Processing Systems*, 33:6476–6489.

Jacob Devlin Ming-Wei Chang Kenton and Lee Kristina Toutanova. 2019. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT*, pages 4171–4186.

Urvashi Khandelwal, Omer Levy, Dan Jurafsky, Luke Zettlemoyer, and Mike Lewis. 2019. Generalization through memorization: Nearest neighbor language models. In *International Conference on Learning Representations*.

Yunsu Kim, Duc Thanh Tran, and Hermann Ney. 2019. When and why is document-level context useful in neural machine translation? In *Proceedings of the Fourth Workshop on Discourse in Machine Translation (DiscoMT 2019)*, pages 24–34.

Diederik P Kingma and Jimmy Ba. 2014. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*.

Nikita Kitaev, Lukasz Kaiser, and Anselm Levskaya. 2019. Reformer: The efficient transformer. In *International Conference on Learning Representations*.

Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut. 2019. Albert: A lite bert for self-supervised learning of language representations. In *International Conference on Learning Representations*.

Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*.

Matt Mahoney. 2011. Large text compression benchmark.

Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2016. Pointer sentinel mixture models. In *International Conference on Learning Representations*.

Hao Peng, Jungo Kasai, Nikolaos Pappas, Dani Yogatama, Zhaofeng Wu, Lingpeng Kong, Roy Schwartz, and Noah A Smith. 2022a. Abc: Attention with bounded-memory control. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 7469–7483.

Hao Peng, Nikolaos Pappas, Dani Yogatama, Roy Schwartz, Noah Smith, and Lingpeng Kong. 2022b. Random feature attention. In *International Conference on Learning Representations*.

Michał Pietruszka, Łukasz Borchmann, and Łukasz Garncarek. 2022. Sparsifying transformer models with trainable representation pooling. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 8616–8633.

Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *The Journal of Machine Learning Research*, 21(1):5485–5551.

Aurko Roy, Mohammad Saffar, Ashish Vaswani, and David Grangier. 2021. Efficient content-based sparse attention with routing transformers. *Transactions of the Association for Computational Linguistics*, 9:53–68.

Sainbayar Sukhbaatar, Édouard Grave, Piotr Bojanowski, and Armand Joulin. 2019. Adaptive attention span in transformers. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 331–335.

Sainbayar Sukhbaatar, Da Ju, Spencer Poff, Stephen Roller, Arthur Szlam, Jason Weston, and Angela Fan. 2021. Not all memories are created equal: Learning to forget by expiring. In *International Conference on Machine Learning*, pages 9902–9912. PMLR.

Yi Tay, Mostafa Dehghani, Dara Bahri, and Donald Metzler. 2022. Efficient transformers: A survey. *ACM Computing Surveys*, 55(6):1–28.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems*, 30.

Apoorv Vyas, Angelos Katharopoulos, and François Fleuret. 2020. Fast transformers with clustered attention. *Advances in Neural Information Processing Systems*, 33:21665–21674.

Sinong Wang, Belinda Z Li, Madian Khabsa, Han Fang, and Hao Ma. 2020. Linformer: Self-attention with linear complexity. *arXiv preprint arXiv:2006.04768*.

Lesly Miculicich Werlen, Dhananjay Ram, Nikolaos Pappas, and James Henderson. 2018. Document-level neural machine translation with hierarchical attention networks. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2947–2954.

Lin Zheng, Chong Wang, and Lingpeng Kong. 2022a. Linear complexity randomized self-attention mechanism. In *International Conference on Machine Learning*, pages 27011–27041. PMLR.

Lin Zheng, Jianbo Yuan, Chong Wang, and Lingpeng Kong. 2022b. Efficient attention via control variates. In *The Eleventh International Conference on Learning Representations*.

Wangchunshu Zhou, Yuchen Eleanor Jiang, Peng Cui, Tiannan Wang, Zhenxin Xiao, Yifan Hou, Ryan Cotterell, and Mrinmaya Sachan. 2023. Recurrentgpt: Interactive generation of (arbitrarily) long text. *arXiv preprint arXiv:2305.13304*.

A Dataset and Evaluation Metrics

WikiText-103 (Merity et al., 2016) is a commonly used word-level language modeling benchmark. It has an average length of 3.6 thousand tokens per article and includes 28 thousand Wikipedia articles. This word-level dataset has a vocabulary size of around 260K. We use the same data pre-processing setting in Dai et al. (2019) for this dataset. We use `perplexity` as our metric.

Enwik8 (Mahoney, 2011) is a character-level language modeling benchmark. This dataset contains 100M unprocessed Wikipedia characters. The train set, dev set, and test set include 80M, 10M, and 10M characters separately. `enwik8` has no pre-processing stage and is directly used. `bpc` (bit per character) is defined as an evaluation metric and we report results on both the dev set and test set.

B Training Configurations

Since we do inference experiments based on a trained model, we separately train two Transformer-XL models for `WikiText-103` and `enwik8`. For the training stage, we use Adam (Kingma and Ba, 2014) to optimize with a batch size=60, learning rate=2.5e-4, target length=150, memory length=150, and a cosine scheduler without warmup steps.

When it comes to a different dataset, we use different Transformer-XL architecture. For `WikiText-103`, we use a 16-layer transformer architecture with 10 heads, 410 hid dim, 0.1 dropout ratio, 0.0 attention dropout ratio, 2100 inner dim, and adaptive softmax mechanism. For `enwik8`, we propose a 12-layer transformer architecture with 8 heads, 512 hid dim, 0.1 dropout ratio, 0.0 attention dropout ratio, and 2048 inner dim. Both models are trained for 350K steps.

A batch size=10 and target length=150 are fixed for all inference experiments to avoid unfair comparison. All experiments including training and inference are conducted using 4 2080Ti GPUs. It takes 280 GPU hours to train the `enwik8` model checkpoint. It takes 61 GPU hours to train the `WikiText-103` model checkpoint.

C Relative Position Embedding

Concerning positional encodings, we maintain the same results with Transformer-XL. The positional encodings include learnable parameters of $\boldsymbol{R}_{i-j}$, $\boldsymbol{u}$, and $\boldsymbol{v}$. Typically, $\boldsymbol{R}_{i-j}$ is derived from a learnable r network included in the model. The advantage of using this design when computing the attention score is that it avoids temporal confusion caused by indexing the same position and considers the relative distance between two tokens. The formula

for attention score calculation with relative position embedding can be written as:

$$\begin{aligned} \boldsymbol{A}_{i,j}^{xl} = & \boldsymbol{X}_i^\top \boldsymbol{W}_q^\top \boldsymbol{W}_k^E \boldsymbol{X}_j + \boldsymbol{X}_i^\top \boldsymbol{W}_q^\top \boldsymbol{W}_k^R \boldsymbol{R}_{i-j} \\ & + \boldsymbol{u}^\top \boldsymbol{W}_k^E \boldsymbol{X}_j + \boldsymbol{v}^\top \boldsymbol{W}_k^R \boldsymbol{R}_{i-j} \end{aligned} \tag{7}$$

Moreover, after doing ablation studies on relative position embedding, we found that $\boldsymbol{R}_{i-j}$ contributes the most to the result and $\boldsymbol{u}$, $\boldsymbol{v}$ only has a small influence on the final performance. The existence of $\boldsymbol{R}_{i-j}$ leads to the exponentially decayed attention probability distribution related to a memory position. As a result, we base our memory selection on the $\boldsymbol{A}_{i,j}^{xl}$ which includes positional information instead of the pure $\boldsymbol{X}_i^\top \boldsymbol{W}_q^\top \boldsymbol{W}_k^E \boldsymbol{X}_j$. To be noticed, all concepts related to $\mathbf{qK}$ are all equipped with position embedding instead of a simple dot product.